

# Session 2.4: Midterm Analytics Pipeline Defense Part 2: Exploratory Findings and Executive Review

PEDAGOGICAL AGENDA

1:2 Delivery Rhythm

## Session Learning Architecture

Structured 3-hour applied computing session designed for business students with zero prior coding experience.

- **Phase 1 (60 Min Lecture):** Computational foundations, business vocabulary, and syntactic mechanics.
- **Phase 2 (120 Min Lab):** Guided hands-on implementation, error diagnosis, and AI co-pilot resolution.
- **Zero Fluff Mandate:** Focus directly on executable Python code, tabular data wrangling, and commercial intelligence.

STRATEGIC OUTCOME

Operational Autonomy

## Business Analytical Competency

Transforming passive spreadsheet users into autonomous programmatic decision makers.

- **Speed & Scale:** Processing datasets exceeding spreadsheet limits with instantaneous vectorized execution.
- **Repeatable Pipelines:** Replacing manual copy-paste workflows with fully automated Python scripts.
- **AI Debugging Literacy:** Leveraging modern IDEs to diagnose and resolve errors independently.

# Core Computational Keywords & Business Meaning

CONCEPT 01

Core Principle

## Syntactic Rigor

Writing clean, self-documenting code that scales across corporate systems.

CONCEPT 02

Core Principle

## Syntactic Rigor

Writing clean, self-documenting code that scales across corporate systems.

CONCEPT 03

Core Principle

## Syntactic Rigor

Writing clean, self-documenting code that scales across corporate systems.

CONCEPT 04

Core Principle

## Syntactic Rigor

Writing clean, self-documenting code that scales across corporate systems.

# Computational Foundations & Business Operations

PILLAR 01

Architecture

## Code Auditability in High-Stakes Environments

In corporate finance and strategic planning, unverified analytics pipelines create catastrophic financial risks. A formal code defense requires students to demonstrate that every calculation traces back to audited primary records.

PILLAR 02

Architecture

## Communicating Analytics to Non-Technical Leadership

Senior leadership rarely reads raw Python code. Analysts must present clear visual exhibits, executive summary metrics, and explicit strategic trade-offs, maintaining code as the verifiable engine underneath.

PILLAR 03

Architecture

## Peer Review and Code Auditing

Evaluating external codebases builds critical debugging and quality assurance skills. Analysts inspect code for hard-coded assumptions, fragile cell dependencies, and unhandled exception paths.

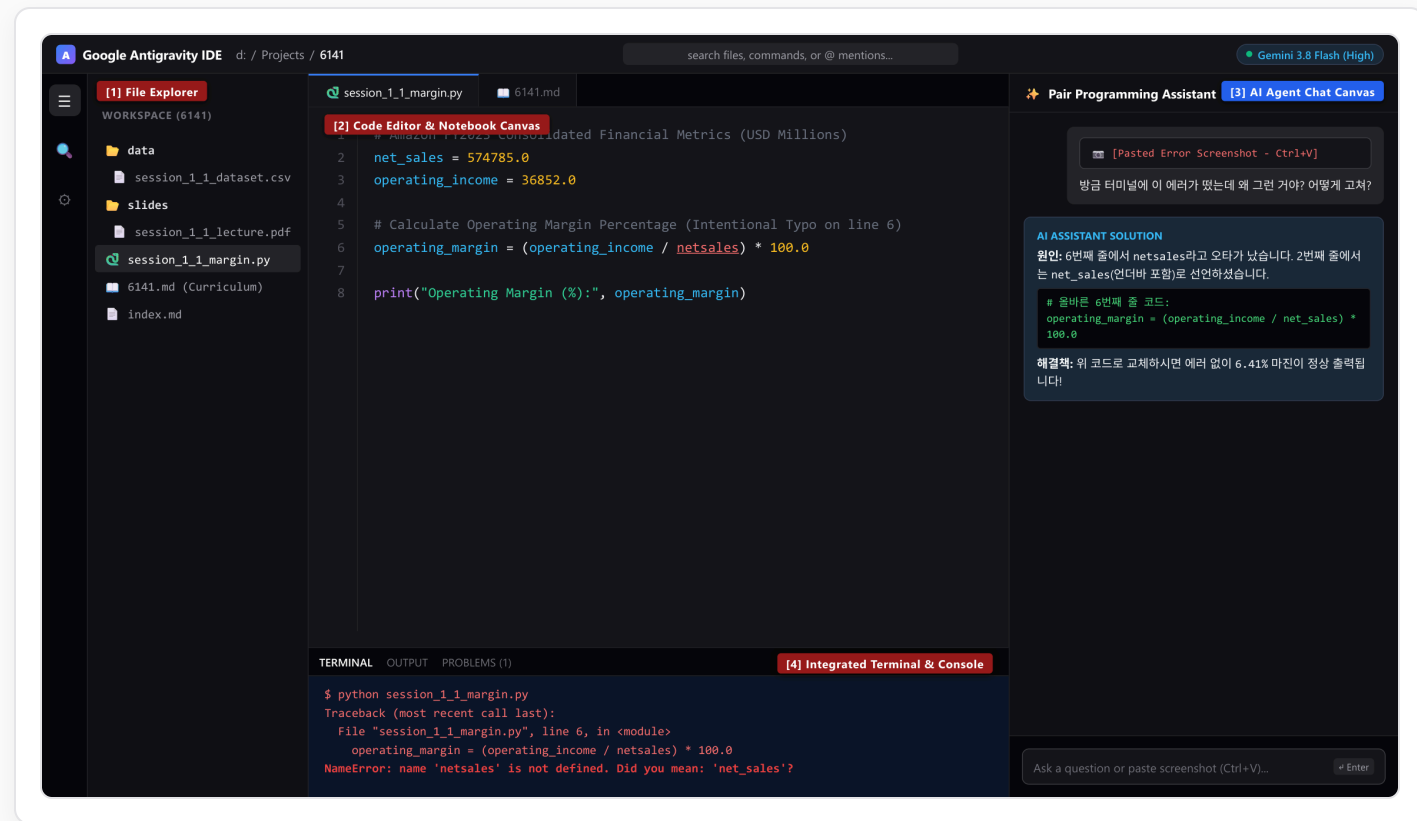
PILLAR 04

Architecture

## Establishing Enterprise Production Baselines

Concluding Module 2 by formalizing baseline operational performance metrics, preparing the analytical foundation for predictive machine learning models in Module 3.

# Google Antigravity IDE Architecture & AI Debugging Protocol



## [1] File Explorer

### Project Hierarchy

Manage datasets in data/, scripts in root, and exported artifacts.

## [2] Code Editor

### Syntax Workspace

Modern syntax highlighting, lint diagnostics, and line-by-line script authoring.

## [3] AI Agent Chat

### AI Pair Programmer

Paste error screenshots directly to inspect root causes and get verified fixes.

## [4] Terminal

### Execution Engine

Run Python scripts (python script.py) and view console logs and tracebacks.

## Zero-Friction AI Error Resolution Protocol

- Capture:** Press Win + Shift + S to clip terminal error traceback.
- Paste:** Press Ctrl + V into AI Chat [3] with "Explain root cause & fix".
- Apply:** Review the explanation and apply verified fix into Editor [2].

# Script Demonstration 1: Script Demonstration 1

session\_2\_4\_demo\_1.pyPython 3.10

```
1  "kw">class="kw">import time
2      "kw">class="kw">import pandas "kw">class="kw">as pd
3
4      start_time = time.time()
5      df = pd."fn">read_csv("data/session_2_4_dataset.csv")
6
7      # Compute platform take rate and cash flows
8      df["Recognized_Revenue_USD_M"] = df["Gross_Merchandise_Value_USD_M"] * (df
9      df["Cash_Flow_Conversion_Pct"] = (df["Operating_Cash_Flow_USD_M"] / df["Re
10
11      elapsed_time = time.time() - start_time
12      "fn">print(f"Pipeline Executed ">class="kw">in {elapsed_time:.4f} seconds
```

\$ python session\_2\_4\_demo\_1.py [Exit 0]

1 Syntactic Structure

Importing production libraries and defining clean analytical data structures.

2 Vectorized Transformation

Applying deterministic operations across the full dataset without fragile cell references.

3 Console Verification

Emitting formatted business KPIs to the terminal for auditability and compliance.

# Script Demonstration 2: Script Demonstration 2

session\_2\_4\_demo\_2.pyPython 3.10

1

segment\_audit = df."fn">groupby("Market\_Segment")."fn">agg(  
2Total\_GMV\_USD\_M=("Gross\_Merchandise\_Value\_USD\_M", "sum"),  
3Total\_Revenue\_USD\_M=("Recognized\_Revenue\_USD\_M", "sum"),  
4Total\_Operating\_Cash\_Flow\_USD\_M=("Operating\_Cash\_Flow\_USD\_M", "sum"),  
5Mean\_Take\_Rate=("Platform\_Take\_Rate\_Pct", "mean")  
6)  
7"fn">print("Geographic Cash Flow Conversion Audit:\\n", segment\_audit)

\$ python session\_2\_4\_demo\_2.py [Exit 0]

1 Syntactic Structure

Importing production libraries and defining clean analytical data structures.

2 Vectorized Transformation

Applying deterministic operations across the full dataset without fragile cell references.

3 Console Verification

Emitting formatted business KPIs to the terminal for auditability and compliance.

Applied Code Walkthrough | Style 2 Harvard Academic Standard

Executable Python Script

Slide 6 / 8

# Script Demonstration 3: Script Demonstration 3

session\_2\_4\_demo\_3.pyPython 3.10

```
1  "kw">class="kw">import hashlib
2
3      summary_str = segment_audit.to_string()
4      audit_hash = hashlib.sha256(summary_str.encode("utf-8")).hexdigest()[:12]
5      segment_audit.to_csv("data/midterm_defense_summary.csv")
6      "fn">print(f"Audited Executive Summary Exported. Verification Hash: {audit_

$ python session_2_4_demo_3.py [Exit 0]
```

1 Syntactic Structure

Importing production libraries and defining clean analytical data structures.

2 Vectorized Transformation

Applying deterministic operations across the full dataset without fragile cell references.

3 Console Verification

Emitting formatted business KPIs to the terminal for auditability and compliance.

# Interactive Two-Tier Lab Protocol & AI Diagnosis

## EXERCISE 1

Guided Lab

### Foundational Implementation

Step-by-step guided practice establishing baseline syntax mastery in Antigravity IDE.

- **Step 1:** Ingest dataset from `data/session_2_4_dataset.csv`.
- **Step 2:** Implement core analytical functions and compute primary business metrics.
- **Step 3:** Format console outputs and verify calculations against baseline ledger totals.
- **Step 4:** Run in Terminal [4] and confirm clean execution with exit code 0.

## EXERCISE 2+

Coding Challenge

### Advanced Scripting & AI Debugging

Applied programming challenge expanding pipeline logic and resolving intentional exceptions.

- **Task 1 (Pipeline Expansion):** Add secondary business unit logic and multi-tier calculations.
- **Task 2 (Deliberate Error & AI Capture):** Introduce intentional syntax or type error, capture traceback with `Win + Shift + S`, and diagnose via AI Chat [3].
- **Task 3 (Verified Production Run):** Apply verified fix, export audit output, and confirm zero errors with return code 0.